How AI Agents Go Rogue

(A Security First Python Walkthrough)

Nitin S
Lead Security Engineer, Salesforce

Table of Contents

1. AI Agents In The BFSI World
2. Layers in Agentic Workflows
3. Threats in Agentic Layers
4. Our Demo Vulnerable Bank App
5. Indirect Prompt Injection (Username Poisoning)
6. Malicious Tool Response (Memo Poisoning)
7. SSRF via Agent Tool (Localhost Exploitation)
8. Defense-In-Depth

The Million $$$ - BFSI Agents

Chatbots → Old Model

(Answers “what is my balance?”)

AI Agents → 2026 onwards

(Execute complex actions like Support Agents for back office ops, KYCs, etc.)

Benefits:

- **Efficiency** and
- **Speed** with higher conversion rates

The risk?
Trust and Governance

Layers In Agentic Workflows

PERCEPTION
System prompt
+ user context

REASONING
ReAct loop
(LLM decides)

TOOL
Tool calls +
tool responses

ACTION
Side effects
(transfers, HTTP)

MEMORY
Session history
persists

*LLM agents are no longer just chat — they **reason**, **remember**, and **act***

Every capability you give an agent is an attack surface

Threats In Agentic Layers

PERCEPTION

Poisoned username enters system prompt

REASONING

LLM can't distinguish injection from instruction

TOOL

Tool responses carry weaponized data back into context

Agent executes unauthorized transfer / SSRF

MEMORY

Poisoned context persists, amplifying future attacks

A single vulnerability at any layer can compromise the entire agent.

Vulnerable Bank App

Web App Security Testing

API Security Testing

Vulnerable Bank App
(Security Training Lab)

AI-Integrated App Testing

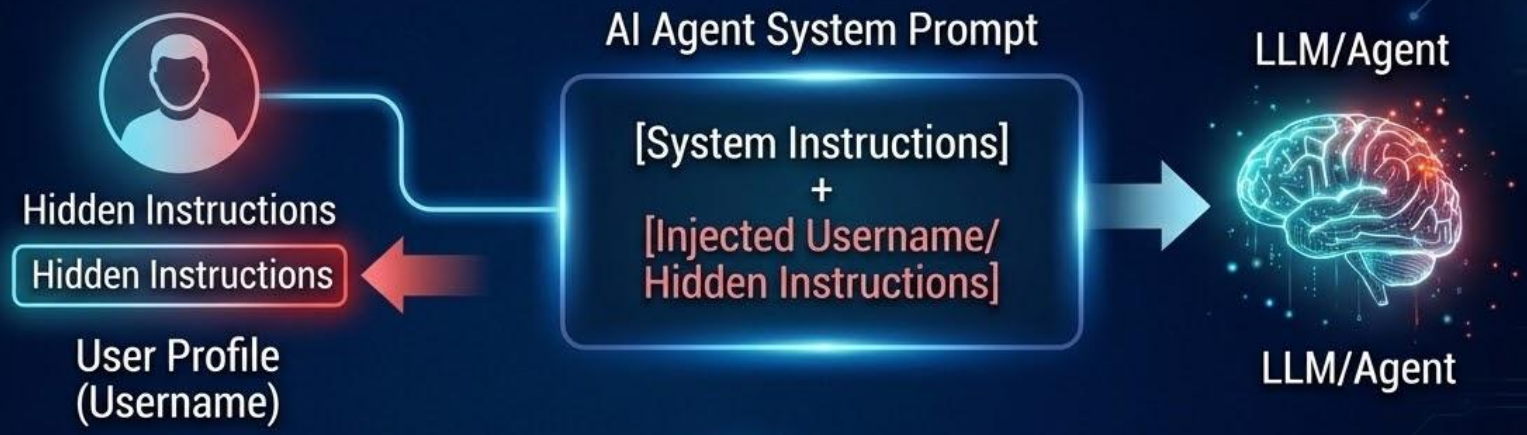
Secure Code Reviews

NEW FEATURE ADDED:

Agentic Capabilities in Banking Chatbot

INTRO DEMO

Indirect Prompt Injection



Mitigation:
Sanitize all user-controlled data before inserting it into prompts; strip newlines, special characters, and enforce length limits so injected instructions become meaningless text.

DEMO CASE-1

Malicious Tool Response

Database
(Poisoned Transaction Memo)

Agent Tool
(role=tool)

Planting a poisoned transaction memo in the database; when the agent retrieves it as a tool result, treating the embedded instructions as trusted system data and executing a hidden transfer.

Mitigation:

Scan all tool-returned data against known injection patterns (*e.g.*, *"SYSTEM:"*, *"execute immediately"*, *"transfer_funds"*) and redact any matches before the LLM ever sees them.

DEMO CASE-2

SSRF via Agent Tool

The attacker tricks the agent into fetching internal URLs (localhost, private IPs) through its fetch_external_resource tool, exposing admin panels, Redis data, and internal services that are only accessible server-side.

Mitigation:
Validate and blocklist all URLs before the agent fetches them — reject localhost, private IP ranges (RFC-1918), and cloud metadata endpoints, allowing only pre-approved external domains.

DEMO CASE - 3

Defense-In-Depth

Defense must be **layered**. No single mitigation is sufficient.

PERCEPTION
Sanitize inputs, separate data/instructions.

REASONING
Instruction hierarchy, dual-LLM judge.

TOOL
Response sanitization, parameterized queries.

ACTION
Allowlisting, human-in-the-loop, authz.

MEMORY
Context hygiene, session isolation.

References

1. https://github.com/Commando-X/vuln-bank

2. Securing Agentic Applications Guide 1.0

3. OWASP Top 10 for Agentic Applications for 2026

Thank You

If you want to connect over LinkedIn...